

Tutorial_1_Building_Web_App

Tutorial 1: Building a Web Application from Scratch

Duration: 45-60 minutes **Level:** Beginner **Prerequisites:** Basic programming knowledge, HelixCode installed

Overview

In this tutorial, you'll build a complete RESTful web application using HelixCode's AI assistance. You'll learn:

- Project initialization with Plan Mode
- Code generation with AI
- Database schema design
- API endpoint creation
- Testing automation
- Git auto-commit
- Deployment

What We're Building: A Task Management API with user authentication, CRUD operations, and PostgreSQL backend.

Step 1: Project Setup

1.1 Create Project Directory

```
mkdir task-api
cd task-api
git init
```

1.2 Initialize HelixCode Session

```
# Start HelixCode interactive session
helixcode

# Create named session
helixcode> session create --name "task-api-build"

# Session created: task-api-build (id: abc123)
```

1.3 Configure AI Provider

```
# List available providers
```

```
helixcode> llm providers
```

```
# Set provider (choose based on your setup)
```

```
helixcode> llm provider set anthropic --model claude-3-5-sonnet-  
latest
```

```
# Verify
```

```
helixcode> llm models
```

```
# ✓ anthropic/claude-3-5-sonnet-latest (active)
```

Step 2: Use Plan Mode for Architecture

2.1 Generate Project Plan

```
helixcode> plan "Create a REST API in Go for task management  
with:
```

- User authentication (JWT)
- CRUD operations for tasks
- PostgreSQL database
- Clean architecture (handler/service/repository layers)
- Comprehensive error handling
- Unit tests
- Docker support"

HelixCode Output:

 Plan Generated (ID: plan-001)

```
## Project: Task Management REST API
```

```
### Phase 1: Project Structure & Dependencies
```

- Initialize Go module
- Create directory structure (cmd/, internal/, pkg/)
- Add dependencies (gin, pgx, jwt-go)
- Create Dockerfile and docker-compose.yml

```
### Phase 2: Database Layer
```

- Define schema (users, tasks tables)
- Create migrations
- Implement repository layer
- Add connection pooling

```
### Phase 3: Service Layer
```

- User authentication service
- Task CRUD service
- Error handling middleware
- Input validation

Phase 4: API Layer

- HTTP handlers (gin)
- JWT middleware
- Request/response models
- Route configuration

Phase 5: Testing

- Unit tests for services
- Integration tests for API
- Test fixtures and helpers

Phase 6: Deployment

- Docker containerization
- Environment configuration
- CI/CD pipeline

Estimated time: 2-3 hours with AI assistance

Complexity: Medium

Options generated: 3

2.2 Review Options

```
helixcode> plan options plan-001
```

```
# Option 1: Standard REST API (recommended)
```

- ```
- Gin framework
- PostgreSQL with pgx
- JWT authentication
- Clean architecture
```

```
Option 2: Microservices Architecture
```

- ```
# - Separate auth service  
# - gRPC communication  
# - Event-driven  
# - More complex
```

```
# Option 3: Serverless
```

- ```
- AWS Lambda
- DynamoDB
- API Gateway
- Cloud-native
```

```
helixcode> plan select plan-001 option-1
```

## 2.3 Execute Plan

```
helixcode> plan execute plan-001
```

```
HelixCode will now execute each phase step-by-step
You can approve each step or let it run automatically
```

---

## Step 3: Project Structure Generation

### 3.1 Initialize Go Module

**HelixCode generates:**

```
Automatic execution
$ go mod init task-api
$ go mod tidy
```

**Created go.mod:**

```
module task-api

go 1.26

require (
 github.com/gin-gonic/gin v1.9.1
 github.com/golang-jwt/jwt/v5 v5.0.0
 github.com/jackc/pgx/v5 v5.4.3
 github.com/google/uuid v1.3.1
 golang.org/x/crypto v0.14.0
)
```

### 3.2 Directory Structure

**HelixCode creates:**

```
task-api/
├── cmd/
│ └── server/
│ └── main.go
├── internal/
│ ├── auth/
│ │ ├── jwt.go
│ │ ├── middleware.go
│ │ └── service.go
│ ├── task/
│ │ ├── handler.go
│ │ ├── service.go
│ │ └── repository.go
│ ├── user/
│ │ ├── handler.go
│ │ ├── service.go
│ │ └── repository.go
│ └── database/
│ └── postgres.go
```

```
├── server/
│ └── server.go
├── pkg/
│ ├── models/
│ │ ├── user.go
│ │ └── task.go
│ └── errors/
│ └── errors.go
├── migrations/
│ ├── 001_create_users_table.sql
│ └── 002_create_tasks_table.sql
├── tests/
│ ├── integration/
│ └── unit/
├── docker-compose.yml
├── Dockerfile
├── .env.example
├── .gitignore
└── README.md
```

---

## Step 4: Database Schema Design

### 4.1 Users Table Migration

**HelixCode** generates migrations/001\_create\_users\_table.sql:

```
-- Create users table
CREATE TABLE IF NOT EXISTS users (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 username VARCHAR(50) UNIQUE NOT NULL,
 email VARCHAR(255) UNIQUE NOT NULL,
 password_hash VARCHAR(255) NOT NULL,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT
 CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Create index on email for fast lookups
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_username ON users(username);

-- Trigger to update updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
 NEW.updated_at = CURRENT_TIMESTAMP;
 RETURN NEW;
END;
$$ language 'plpgsql';
```

```
CREATE TRIGGER update_users_updated_at BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

## 4.2 Tasks Table Migration

HelixCode generates migrations/002\_create\_tasks\_table.sql:

```
-- Create tasks table
CREATE TABLE IF NOT EXISTS tasks (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
 title VARCHAR(255) NOT NULL,
 description TEXT,
 status VARCHAR(20) NOT NULL DEFAULT 'pending',
 priority VARCHAR(20) DEFAULT 'medium',
 due_date TIMESTAMP WITH TIME ZONE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT
 CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT
 CURRENT_TIMESTAMP,

 CONSTRAINT valid_status CHECK (status IN ('pending',
 'in_progress', 'completed', 'cancelled')),
 CONSTRAINT valid_priority CHECK (priority IN ('low',
 'medium', 'high', 'urgent'))
);

-- Indexes for performance
CREATE INDEX idx_tasks_user_id ON tasks(user_id);
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_due_date ON tasks(due_date);

-- Update trigger
CREATE TRIGGER update_tasks_updated_at BEFORE UPDATE ON tasks
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

---

## Step 5: Implement Models

### 5.1 User Model

HelixCode generates pkg/models/user.go:

```
package models

import (
 "time"
 "github.com/google/uuid"
```

```

)

// User represents a user in the system
type User struct {
 ID uuid.UUID `json:"id" db:"id"`
 Username string `json:"username" db:"username"`
 Email string `json:"email" db:"email"`
 PasswordHash string `json:"-" db:"password_hash"`
 CreatedAt time.Time `json:"created_at" db:"created_at"`
 UpdatedAt time.Time `json:"updated_at" db:"updated_at"`
}

// UserCreateRequest represents user registration request
type UserCreateRequest struct {
 Username string `json:"username"
 binding:"required,min=3,max=50"`
 Email string `json:"email" binding:"required,email"`
 Password string `json:"password" binding:"required,min=8"`
}

// UserLoginRequest represents login request
type UserLoginRequest struct {
 Email string `json:"email" binding:"required,email"`
 Password string `json:"password" binding:"required"`
}

// UserResponse represents user response (without sensitive data)
type UserResponse struct {
 ID uuid.UUID `json:"id"`
 Username string `json:"username"`
 Email string `json:"email"`
 CreatedAt time.Time `json:"created_at"`
}

// ToResponse converts User to UserResponse
func (u *User) ToResponse() UserResponse {
 return UserResponse{
 ID: u.ID,
 Username: u.Username,
 Email: u.Email,
 CreatedAt: u.CreatedAt,
 }
}

```

## 5.2 Task Model

**HelixCode** generates `pkg/models/task.go`:

```

package models

import (
 "time"
 "github.com/google/uuid"
)

// TaskStatus represents the status of a task
type TaskStatus string

const (
 StatusPending TaskStatus = "pending"
 StatusInProgress TaskStatus = "in_progress"
 StatusCompleted TaskStatus = "completed"
 StatusCancelled TaskStatus = "cancelled"
)

// TaskPriority represents the priority level
type TaskPriority string

const (
 PriorityLow TaskPriority = "low"
 PriorityMedium TaskPriority = "medium"
 PriorityHigh TaskPriority = "high"
 PriorityUrgent TaskPriority = "urgent"
)

// Task represents a task
type Task struct {
 ID uuid.UUID `json:"id" db:"id"`
 UserID uuid.UUID `json:"user_id" db:"user_id"`
 Title string `json:"title" db:"title"`
 Description string `json:"description" db:"description"`
 Status TaskStatus `json:"status" db:"status"`
 Priority TaskPriority `json:"priority" db:"priority"`
 DueDate *time.Time `json:"due_date,omitempty" db:"due_date"`
 CreatedAt time.Time `json:"created_at" db:"created_at"`
 UpdatedAt time.Time `json:"updated_at" db:"updated_at"`
}

// TaskCreateRequest represents task creation request
type TaskCreateRequest struct {
 Title string `json:"title" binding:"required,min=1,max=255"`
 Description string `json:"description"`
}

```



```

 Priority TaskPriority `json:"priority"
 binding:"omitempty,oneof=low medium high urgent"`
 DueDate *time.Time `json:"due_date,omitempty"`
 }

 // TaskUpdateRequest represents task update request
 type TaskUpdateRequest struct {
 Title *string `json:"title,omitempty"
 binding:"omitempty,min=1,max=255"`
 Description *string `json:"description,omitempty"`
 Status *TaskStatus `json:"status,omitempty"
 binding:"omitempty,oneof=pending in_progress completed
 cancelled"`
 Priority *TaskPriority `json:"priority,omitempty"
 binding:"omitempty,oneof=low medium high urgent"`
 DueDate *time.Time `json:"due_date,omitempty"`
 }

 // TaskListFilters represents task listing filters
 type TaskListFilters struct {
 Status *TaskStatus `form:"status"`
 Priority *TaskPriority `form:"priority"`
 Page int `form:"page,default=1"`
 PerPage int `form:"per_page,default=20"`
 }

```

---

## Step 6: Implement Repository Layer

Ask HelixCode:

```
helixcode> generate "Implement the task repository with CRUD
 operations using pgx/v5"
```

HelixCode generates internal/task/repository.go:

```

package task

import (
 "context"
 "fmt"
 "github.com/google/uuid"
 "github.com/jackc/pgx/v5"
 "github.com/jackc/pgx/v5/pgxpool"
 "task-api/pkg/models"
 "task-api/pkg/errors"
)

// Repository handles task data persistence

```

```

type Repository struct {
 db *pgxpool.Pool
}

// NewRepository creates a new task repository
func NewRepository(db *pgxpool.Pool) *Repository {
 return &Repository{db: db}
}

// Create inserts a new task
func (r *Repository) Create(ctx context.Context, task
 *models.Task) error {
 query := `
 INSERT INTO tasks (id, user_id, title, description,
 status, priority, due_date)
 VALUES ($1, $2, $3, $4, $5, $6, $7)
 RETURNING created_at, updated_at
 `

 err := r.db.QueryRow(ctx, query,
 task.ID,
 task.UserID,
 task.Title,
 task.Description,
 task.Status,
 task.Priority,
 task.DueDate,
).Scan(&task.CreatedAt, &task.UpdatedAt)

 if err != nil {
 return fmt.Errorf("failed to create task: %w", err)
 }

 return nil
}

// GetByID retrieves a task by ID
func (r *Repository) GetByID(ctx context.Context, id uuid.UUID,
 userID uuid.UUID) (*models.Task, error) {
 query := `
 SELECT id, user_id, title, description, status, priority,
 due_date, created_at, updated_at
 FROM tasks
 WHERE id = $1 AND user_id = $2
 `

 task := &models.Task{}
 err := r.db.QueryRow(ctx, query, id, userID).Scan(

```

```

 &task.ID,
 &task.UserID,
 &task.Title,
 &task.Description,
 &task.Status,
 &task.Priority,
 &task.DueDate,
 &task.CreatedAt,
 &task.UpdatedAt,
)
}

if err != nil {
 if err == pgx.ErrNoRows {
 return nil, errors.ErrTaskNotFound
 }
 return nil, fmt.Errorf("failed to get task: %w", err)
}

return task, nil
}

// List retrieves tasks for a user with filters
func (r *Repository) List(ctx context.Context, userID uuid.UUID,
 filters *models.TaskListFilters) ([]*models.Task, int,
 error) {
 // Build dynamic query based on filters
 query := `
 SELECT id, user_id, title, description, status, priority,
 due_date, created_at, updated_at
 FROM tasks
 WHERE user_id = $1
 `

 countQuery := `SELECT COUNT(*) FROM tasks WHERE user_id = $1`
 args := []interface{}{userID}
 argPos := 2

 // Apply filters
 if filters.Status != nil {
 query += fmt.Sprintf(" AND status = %d", argPos)
 countQuery += fmt.Sprintf(" AND status = %d", argPos)
 args = append(args, *filters.Status)
 argPos++
 }

 if filters.Priority != nil {
 query += fmt.Sprintf(" AND priority = %d", argPos)
 countQuery += fmt.Sprintf(" AND priority = %d", argPos)
 args = append(args, *filters.Priority)
 }
}

```

```

 argPos++
 }

 // Get total count
 var total int
 err := r.db.QueryRow(ctx, countQuery, args...).Scan(&total)
 if err != nil {
 return nil, 0, fmt.Errorf("failed to count tasks: %w",
 err)
 }

 // Add pagination
 offset := (filters.Page - 1) * filters.PerPage
 query += fmt.Sprintf(" ORDER BY created_at DESC LIMIT %d
 OFFSET %d", argPos, argPos+1)
 args = append(args, filters.PerPage, offset)

 // Execute query
 rows, err := r.db.Query(ctx, query, args...)
 if err != nil {
 return nil, 0, fmt.Errorf("failed to list tasks: %w",
 err)
 }
 defer rows.Close()

 tasks := make([]*models.Task, 0)
 for rows.Next() {
 task := &models.Task{}
 err := rows.Scan(
 &task.ID,
 &task.UserID,
 &task.Title,
 &task.Description,
 &task.Status,
 &task.Priority,
 &task.DueDate,
 &task.CreatedAt,
 &task.UpdatedAt,
)
 if err != nil {
 return nil, 0, fmt.Errorf("failed to scan task: %w",
 err)
 }
 tasks = append(tasks, task)
 }

 return tasks, total, nil
}

```

```

// Update updates a task
func (r *Repository) Update(ctx context.Context, task
 *models.Task) error {
 query := `
 UPDATE tasks
 SET title = $1, description = $2, status = $3, priority =
 $4, due_date = $5
 WHERE id = $6 AND user_id = $7
 RETURNING updated_at
 `

 err := r.db.QueryRow(ctx, query,
 task.Title,
 task.Description,
 task.Status,
 task.Priority,
 task.DueDate,
 task.ID,
 task.UserID,
).Scan(&task.UpdatedAt)

 if err != nil {
 if err == pgx.ErrNoRows {
 return errors.ErrTaskNotFound
 }
 return fmt.Errorf("failed to update task: %w", err)
 }

 return nil
}

// Delete deletes a task
func (r *Repository) Delete(ctx context.Context, id uuid.UUID,
 userID uuid.UUID) error {
 query := `DELETE FROM tasks WHERE id = $1 AND user_id = $2`

 result, err := r.db.Exec(ctx, query, id, userID)
 if err != nil {
 return fmt.Errorf("failed to delete task: %w", err)
 }

 if result.RowsAffected() == 0 {
 return errors.ErrTaskNotFound
 }

 return nil
}

```

---

## Step 7: Implement Service Layer

```
helixcode> generate "Implement task service layer with business logic"
```

**HelixCode** generates `internal/task/service.go`:

```
package task

import (
 "context"
 "github.com/google/uuid"
 "task-api/pkg/models"
 "task-api/pkg/errors"
)

// Service handles task business logic
type Service struct {
 repo *Repository
}

// NewService creates a new task service
func NewService(repo *Repository) *Service {
 return &Service{repo: repo}
}

// CreateTask creates a new task
func (s *Service) CreateTask(ctx context.Context, userID
 uuid.UUID, req *models.TaskCreateRequest) (*models.Task,
 error) {
 // Validate request
 if req.Title == "" {
 return nil, errors.ErrInvalidInput("title cannot be
 empty")
 }

 // Set defaults
 priority := req.Priority
 if priority == "" {
 priority = models.PriorityMedium
 }

 // Create task
 task := &models.Task{
 ID: uuid.New(),
 UserID: userID,
 Title: req.Title,
 Description: req.Description,
 Status: models.StatusPending,
```

```

 Priority: priority,
 DueDate: req.DueDate,
 }

 err := s.repo.Create(ctx, task)
 if err != nil {
 return nil, err
 }

 return task, nil
}

// GetTask retrieves a task
func (s *Service) GetTask(ctx context.Context, id uuid.UUID,
 userID uuid.UUID) (*models.Task, error) {
 return s.repo.GetByID(ctx, id, userID)
}

// ListTasks lists tasks with filters
func (s *Service) ListTasks(ctx context.Context, userID
 uuid.UUID, filters *models.TaskListFilters)
 ([]*models.Task, int, error) {
 // Validate pagination
 if filters.Page < 1 {
 filters.Page = 1
 }
 if filters.PerPage < 1 || filters.PerPage > 100 {
 filters.PerPage = 20
 }

 return s.repo.List(ctx, userID, filters)
}

// UpdateTask updates a task
func (s *Service) UpdateTask(ctx context.Context, id uuid.UUID,
 userID uuid.UUID, req *models.TaskUpdateRequest)
 (*models.Task, error) {
 // Get existing task
 task, err := s.repo.GetByID(ctx, id, userID)
 if err != nil {
 return nil, err
 }

 // Apply updates
 if req.Title != nil {
 task.Title = *req.Title
 }
 if req.Description != nil {

```

```

 task.Description = *req.Description
 }
 if req.Status != nil {
 task.Status = *req.Status
 }
 if req.Priority != nil {
 task.Priority = *req.Priority
 }
 if req.DueDate != nil {
 task.DueDate = req.DueDate
 }

 // Save
 err = s.repo.Update(ctx, task)
 if err != nil {
 return nil, err
 }

 return task, nil
}

// DeleteTask deletes a task
func (s *Service) DeleteTask(ctx context.Context, id uuid.UUID,
 userID uuid.UUID) error {
 return s.repo.Delete(ctx, id, userID)
}

```

---

## Step 8: Implement HTTP Handlers

helixcode> generate "Implement Gin HTTP handlers for task API"

**HelixCode** generates internal/task/handler.go:

```

package task

import (
 "net/http"
 "github.com/gin-gonic/gin"
 "github.com/google/uuid"
 "task-api/pkg/models"
 "task-api/pkg/errors"
)

// Handler handles HTTP requests for tasks
type Handler struct {
 service *Service
}

```



```

// NewHandler creates a new task handler
func NewHandler(service *Service) *Handler {
 return &Handler{service: service}
}

// Create handles POST /tasks
func (h *Handler) Create(c *gin.Context) {
 var req models.TaskCreateRequest
 if err := c.ShouldBindJSON(&req); err != nil {
 c.JSON(http.StatusBadRequest, gin.H{"error":
 err.Error()})
 return
 }

 // Get user ID from context (set by auth middleware)
 userID := c.GetString("user_id")
 userUUID, _ := uuid.Parse(userID)

 task, err := h.service.CreateTask(c.Request.Context(),
 userUUID, &req)
 if err != nil {
 c.JSON(http.StatusInternalServerError, gin.H{"error":
 err.Error()})
 return
 }

 c.JSON(http.StatusCreated, task)
}

// Get handles GET /tasks/:id
func (h *Handler) Get(c *gin.Context) {
 id, err := uuid.Parse(c.Param("id"))
 if err != nil {
 c.JSON(http.StatusBadRequest, gin.H{"error": "invalid
 task ID"})
 return
 }

 userID := c.GetString("user_id")
 userUUID, _ := uuid.Parse(userID)

 task, err := h.service.GetTask(c.Request.Context(), id,
 userUUID)
 if err != nil {
 if err == errors.ErrTaskNotFound {
 c.JSON(http.StatusNotFound, gin.H{"error": "task not
 found"})
 return
 }
 }
}

```

```

 }
 c.JSON(http.StatusInternalServerError, gin.H{"error":
 err.Error()})
 return
}

c.JSON(http.StatusOK, task)
}

// List handles GET /tasks
func (h *Handler) List(c *gin.Context) {
 var filters models.TaskListFilters
 if err := c.ShouldBindQuery(&filters); err != nil {
 c.JSON(http.StatusBadRequest, gin.H{"error":
 err.Error()})
 return
 }

 userID := c.GetString("user_id")
 userUUID, _ := uuid.Parse(userID)

 tasks, total, err :=
 h.service.ListTasks(c.Request.Context(), userUUID,
 &filters)
 if err != nil {
 c.JSON(http.StatusInternalServerError, gin.H{"error":
 err.Error()})
 return
 }

 c.JSON(http.StatusOK, gin.H{
 "data": tasks,
 "total": total,
 "page": filters.Page,
 "per_page": filters.PerPage,
 })
}

// Update handles PUT /tasks/:id
func (h *Handler) Update(c *gin.Context) {
 id, err := uuid.Parse(c.Param("id"))
 if err != nil {
 c.JSON(http.StatusBadRequest, gin.H{"error": "invalid
 task ID"})
 return
 }

 var req models.TaskUpdateRequest

```

```

 if err := c.ShouldBindJSON(&req); err != nil {
 c.JSON(http.StatusBadRequest, gin.H{"error":
 err.Error()})
 return
 }

 userID := c.GetString("user_id")
 userUUID, _ := uuid.Parse(userID)

 task, err := h.service.UpdateTask(c.Request.Context(), id,
 userUUID, &req)
 if err != nil {
 if err == errors.ErrTaskNotFound {
 c.JSON(http.StatusNotFound, gin.H{"error": "task not
 found"})
 return
 }
 c.JSON(http.StatusInternalServerError, gin.H{"error":
 err.Error()})
 return
 }

 c.JSON(http.StatusOK, task)
}

// Delete handles DELETE /tasks/:id
func (h *Handler) Delete(c *gin.Context) {
 id, err := uuid.Parse(c.Param("id"))
 if err != nil {
 c.JSON(http.StatusBadRequest, gin.H{"error": "invalid
 task ID"})
 return
 }

 userID := c.GetString("user_id")
 userUUID, _ := uuid.Parse(userID)

 err = h.service.DeleteTask(c.Request.Context(), id, userUUID)
 if err != nil {
 if err == errors.ErrTaskNotFound {
 c.JSON(http.StatusNotFound, gin.H{"error": "task not
 found"})
 return
 }
 c.JSON(http.StatusInternalServerError, gin.H{"error":
 err.Error()})
 return
 }
}

```

```

 c.JSON(http.StatusNoContent, nil)
 }

 // RegisterRoutes registers task routes
 func (h *Handler) RegisterRoutes(router *gin.RouterGroup) {
 tasks := router.Group("/tasks")
 {
 tasks.POST("", h.Create)
 tasks.GET("", h.List)
 tasks.GET("/:id", h.Get)
 tasks.PUT("/:id", h.Update)
 tasks.DELETE("/:id", h.Delete)
 }
 }
}

```

---

## Step 9: Implement Authentication

```

helixcode> generate
 "Implement JWT authentication middleware with login and
 registration"

```

**HelixCode generates authentication, JWT middleware, and user handlers** (code omitted for brevity - see full implementation in generated files).

---

## Step 10: Create Main Server

```

helixcode> generate "Create main.go that wires everything
 together with Gin router"

```

**HelixCode generates** cmd/server/main.go:

```

package main

import (
 "context"
 "log"
 "os"
 "os/signal"
 "syscall"
 "time"

 "github.com/gin-gonic/gin"
 "github.com/jackc/pgx/v5/pgxpool"
 "task-api/internal/auth"
 "task-api/internal/task"
 "task-api/internal/user"

```

)

```
func main() {
 // Load configuration
 dbURL := os.Getenv("DATABASE_URL")
 if dbURL == "" {
 dbURL = "postgres://localhost/task_api?sslmode=disable"
 }

 jwtSecret := os.Getenv("JWT_SECRET")
 if jwtSecret == "" {
 log.Fatal("JWT_SECRET environment variable is required")
 }

 // Connect to database
 pool, err := pgxpool.New(context.Background(), dbURL)
 if err != nil {
 log.Fatalf("Unable to connect to database: %v", err)
 }
 defer pool.Close()

 // Initialize repositories
 userRepo := user.NewRepository(pool)
 taskRepo := task.NewRepository(pool)

 // Initialize services
 authService := auth.NewService([]byte(jwtSecret))
 userService := user.NewService(userRepo, authService)
 taskService := task.NewService(taskRepo)

 // Initialize handlers
 userHandler := user.NewHandler(userService)
 taskHandler := task.NewHandler(taskService)

 // Setup router
 router := gin.Default()

 // Public routes
 public := router.Group("/api")
 {
 public.POST("/register", userHandler.Register)
 public.POST("/login", userHandler.Login)
 }

 // Protected routes
 protected := router.Group("/api")
 protected.Use(auth.Middleware(authService))
 {
 taskHandler.RegisterRoutes(protected)
 }
}
```

```

 userHandler.RegisterRoutes(protected)
 }

 // Health check
 router.GET("/health", func(c *gin.Context) {
 c.JSON(200, gin.H{"status": "healthy"})
 })

 // Graceful shutdown
 srv := &http.Server{
 Addr: ":8080",
 Handler: router,
 }

 go func() {
 if err := srv.ListenAndServe(); err != nil && err !=
 http.ErrServerClosed {
 log.Fatalf("listen: %s\n", err)
 }
 }()

 log.Println("Server started on :8080")

 // Wait for interrupt signal
 quit := make(chan os.Signal, 1)
 signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
 <-quit

 log.Println("Shutting down server...")

 ctx, cancel := context.WithTimeout(context.Background(),
 5*time.Second)
 defer cancel()

 if err := srv.Shutdown(ctx); err != nil {
 log.Fatal("Server forced to shutdown:", err)
 }

 log.Println("Server exited")
}

```

---

## Step 11: Auto-Generate Tests

```
helixcode> generate "Create comprehensive unit tests for task
service"
```

**HelixCode** generates tests/unit/task\_service\_test.go with complete test suite.

---

## Step 12: Use Git Auto-Commit

```
HelixCode has been generating code - let's commit it

helixcode> commit --auto

HelixCode analyzes changes and generates:
#
feat: implement complete task management REST API
#
- Add user authentication with JWT
- Implement task CRUD operations
- Add PostgreSQL database layer with migrations
- Create clean architecture (handler/service/repository)
- Add comprehensive error handling
- Implement request validation
- Add pagination and filtering for task listing
- Include unit tests for services
#
🤖 Generated with HelixCode AI
Co-Authored-By: Claude <noreply@anthropic.com>

git push origin main
```

---

## Step 13: Docker Deployment

**HelixCode generated** docker-compose.yml:

```
version: '3.8'

services:
 postgres:
 image: postgres:15
 environment:
 POSTGRES_DB: task_api
 POSTGRES_USER: task_api
 POSTGRES_PASSWORD: password
 ports:
 - "5432:5432"
 volumes:
 - postgres_data:/var/lib/postgresql/data
 - ./migrations:/docker-entrypoint-initdb.d

 api:
 build: .
 ports:
 - "8080:8080"
```

```
environment:
 DATABASE_URL: postgres://task_api:password@postgres:5432/
 task_api?sslmode=disable
 JWT_SECRET: your-secret-key-change-in-production
depends_on:
 - postgres

volumes:
 postgres_data:
```

### Deploy:

```
Build and start
docker-compose up -d

Check logs
docker-compose logs -f api

Run migrations
docker-compose exec api ./migrate

Test API
curl http://localhost:8080/health
```

---

## Step 14: Test the API

### Register User

```
curl -X POST http://localhost:8080/api/register \
 -H "Content-Type: application/json" \
 -d '{
 "username": "testuser",
 "email": "test@example.com",
 "password": "password123"
 }'
```

```
Response:
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"username": "testuser",
"email": "test@example.com",
"created_at": "2025-11-06T10:00:00Z"
}
```

### Login

```
curl -X POST http://localhost:8080/api/login \
 -H "Content-Type: application/json" \
```



```
-d '{
 "email": "test@example.com",
 "password": "password123"
}'

Response:
{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"user": {...}
}

Save token
export TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

## Create Task

```
curl -X POST http://localhost:8080/api/tasks \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $TOKEN" \
 -d '{
 "title": "Build REST API",
 "description": "Complete the task management API",
 "priority": "high",
 "due_date": "2025-12-01T00:00:00Z"
 }'
```

## List Tasks

```
curl -X GET "http://localhost:8080/api/tasks?
 status=pending&page=1&per_page=10" \
 -H "Authorization: Bearer $TOKEN"
```

## Update Task

```
curl -X PUT http://localhost:8080/api/tasks/550e8400-e29b-41d4-
 a716-446655440000 \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $TOKEN" \
 -d '{
 "status": "completed"
 }'
```

---

## Conclusion

**What You've Built:** -  Complete REST API with authentication -  PostgreSQL database with migrations -  Clean architecture (3-layer) -  Comprehensive error handling -  Unit tests -  Docker deployment -  Production-ready code

**Time Saved:** With HelixCode AI assistance, this project took ~45 minutes instead of 6-8 hours!

**Next Steps:** - Add more endpoints (task sharing, comments) - Implement frontend (React/Vue) - Add CI/CD pipeline - Deploy to production (AWS/GCP/Azure)

**HelixCode Features Used:** - ✨ Plan Mode - ✨ Code Generation - ✨ Git Auto-Commit - ✨ LLM Integration (Claude 3.5)

---

**Tutorial Complete!** 🎉

Continue to [Tutorial 2: Refactoring a Large Codebase](#)

## Sources verified

Sources verified 2026-05-29: <https://go.dev/dl/> (go1.26.3 latest stable Go; 1.24 past support) ; project go.mod (root go 1.25.2, inner go 1.26) + CLAUDE.md §3.1 (PostgreSQL 15+).